Neural Networks Augmented with CBR-Style Adaptation

Larry Gates and Caleb Kisby
gatesla@iu.edu & cckisby@iu.edu

Abstract

In this paper, we consider how case-based reasoning can be viewed as a model for integrating domain reasoning in hybrid systems. In particular, case-based reasoning performs domain reasoning (adaptation) on an existing prediction of a different case (retrieval). We suggest a similar architecture for performing adaptation on an existing *neural network* prediction of a nearby case. We implement CHRISTINE, a system that employs adaptation on neural network predicted *ghost cases* to predict the price of used cars. Our results about this novel architecture leave the reader wanting, so we consider several things we should improve before this work is conference-ready. On the other hand, our adaptation significantly improves performance using only three simple rules; we are very proud of our baby!

Introduction

Neural networks (NNs) and knowledge-based artificial intelligence (AI) each have their own strengths and weaknesses. NNs are known for fairly accurate predictions in domains where a pattern can be learned, but perform poorly on structured, knowledge-rich domains. They have the additional advantage of having high accuracy with very little manual effort. Knowledge-based AI, on the other hand, performs well when the rules of a domain are known, but unfortunately requires significant manual effort to codify knowledge in the form of these rules. But we are not forced to make this tradeoff; a recent burst of work in *hybrid systems* suggests ways to combine NNs alongside knowledge-based AI to make up for the deficiencies of the other.

A particularly fruitful front is work on case-based reasoning (CBR) and NN hybrid systems. CBR is a flexible knowledge-based approach that suggests that to solve a problem, we should adapt a similar case we've seen before. Because the CBR process is completely transparent, CBR has been used to supplement NNs with, e.g., explanations and evaluations of confidence. Although explainability is an important feature of these hybrids, CBR also has a performance advantage in some domains. Namely, case adaptation amounts to domain reasoning, which is best-suited for knowledge-rich domains. Our paper focuses on the use of adaptation in CBR to supplement the performance of a NN in such a domain.

The hybrid approach we propose is to consider CBR as a model for how one can do reasoning *on top of* a prior prediction. CBR performs retrieval to obtain a preliminary prediction, and refines this with adaptation (i.e. domain reasoning). In an analogous way, we suggest obtaining NN predictions, and refining these with adaptation. But in order to apply adaptation, the cases we "retrieve" must be different from the problem case. So for this approach to work, we adapt artificial, nearby perturbations of the problem case. We hope that this "NN + adaptation" setup improves on NN performance in the same way that CBR improves on retrieval alone.

The domain we focus on is used car pricing, a surprisingly nontrivial knowledge-rich domain. Car pricing benefits from common rules-of-thumb that are often explicitly mentioned by experts or can be intuited after some practice buying cars. This knowledge seeps into nearly every component of the system we present. In fact, a key feature of our work is that we put in the knowledge-engineering effort to arrive at adaptation rules that work.

Related Work

Neural Net & CBR Hybrid Systems

The approach we present here is one of many ways one can pair a NN alongside CBR. Another major paradigm is the *twin systems* approach, surveyed by (Keane and Kenny 2019). In a twin systems setup, the CBR and NN by definition perform prediction seperately, "side-by-side." Their point is to use CBR to *explain* the predictions of the NN. Our approach differs in that CBR is *pipelined* with the NN with the intention of improving performance.

On the other hand, hybrid systems that pipeline CBR and the NN into a single system typicaly do not give significant attention to case adaptation. For a couple of examples, see (Rodriguez et al. 2006) and (Reategui, Campbell, and Leao 1997).

There are a few hybrid systems combining CBR and NN's that *do* address adaptation: (Policastro, Carvalho, and Delbem 2008) (Liao, Liu, and Chao 2018) and (Leake, Ye, and Crandall 2021), to list just three. But the unifying theme of these works is to use machine learning to *learn* adaptation rules,

Not copyrighted © 2020, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights unreserved. Please call for reservations.

which would otherwise be difficult to acquire. In this paper we instead face the manual effort needed to craft adaptation rules, and apply these rules to the NN's predictions to improve performance.

Adaptation

In this paper, we walk through how we arrived at our case adaptation for the used car pricing domain. But this paper is by no means a tutorial on adaptation. If the reader would like to follow in our footsteps, we highly recommend the following resources. First, the classic book (Kolodner 2014) has an excellent step-by-step discussion of the considerations one should make when doing case adaptation. Additionally, (Munoz-Avila and Cox 2008) surveys adaptation in the context of *planning*, which is exactly the perspective that helps explain adaptation the best (at least in our opinion).

Our Approach

Case-based reasoning itself suggests a path for integrating domain reasoning alongside a NN's predictions. Consider the CBR process: When presented with a problem, we first remember a past experience, and then adapt this experience towards the problem at hand. In particular, CBR consists of at least these two seperable processes: retrieval, followed by adaptation. In this way, we can view CBR as a hybrid systems approach: A CBR system combines a retrieval system alongside domain reasoning (i.e. adaptation).

In order to incorporate domain reasoning in a NN, we can try to architect a similar setup. When faced with a problem, we first look at nearby problems and predict their solutions using the NN. We call these nearby cases *ghost cases*[1]. We can collect these problems in a *ghost base*, which then allows us to do adaptation (i.e. domain reasoning) on the NN's predictions.

Figure 1 illustrates our intuition. Consider problem case $\vec{x}$ with actual solution y and prediction (from the NN) n. Say that the closest ghost case to $\vec{x}$ is $\vec{a}$, and that the NN gives it a prediction of b. We expect $(\vec{a}, b)$ to be close to $(\vec{x}, n)$, since $\vec{a}$ is generated to be close to $\vec{x}$ and b, n are both predictions from the NN. On the other hand, we shouldn't expect b to be any closer to the actual y than the NN's original prediction n. This is alright, since adaptation on $(\vec{a}, b)$ (resulting in b') will take us even further towards the actual solution — and because of the closeness of $\vec{a}$ and $\vec{x}$, we can rest assured that it doesn't take much reasoning to make b' closer to y than n.

We will now explain in detail how we go about generating this ghost base. Then, we will walk through how we do adaptation for the car pricing domain.

Ghost Base Generation

Given a problem case, the ghost base is an artificial set of *close variations* on this case, along with their NN predictions. Generating all *permutations* of a case's attributes is overkill, since we cannot expect every possible permutation to be close to the original case. So we have to alter cases somewhat carefully.

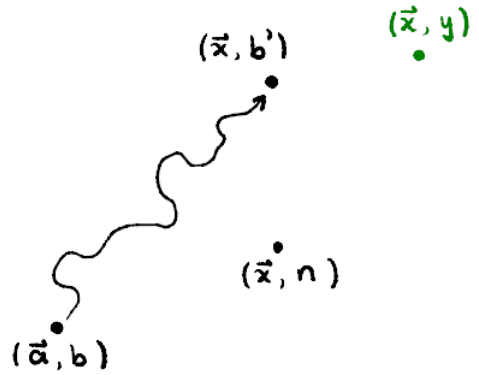

Figure 1: A ghost case $(\vec{a}, b)$ may not be closer to the actual solution y than a NN's prediction n, but we expect that an adapted solution b' will be closer.

We stick with the basic idea of altering one attribute at a time. But doing this naïvely in our car pricing domain, we run into an issue. Namely, we end up generating cars that are impossible: Given a 2008 Honda Accord, we would produce a 2008 Chevy Accord as a candidate ghost case[2]. Any price the NN will predict for these *impossible* cars is unlikely to mean anything, and so we require that our ghost cases are *valid* cars.

We can avoid generating impossible cases by altering *attribute dependency groups* instead of individual attributes. An attribute dependency group collects attributes whose allowed values are constrained by the others. For example, a car's body type determines its number of seats, so we group these two attributes together. All attributes that relate to the engine (e.g. horsepower, cylinders, fuel economy) are put into an engine group. Remaining attributes, such as the car's color and mileage, are assumed to be independent from each other.

Given a problem car, we alter each group one at a time (i.e. each ghost car differs from the original car by at most one group). When altering a group, we constrain the possible choices to only those configurations we know are possible. For example, given the car:

$$\text{2017 Honda Civic} \quad \text{Sedan}, 4 \text{ seats} \quad \text{Frame damaged}$$

a valid ghost case might be the following, where only the body type and maximum seating have changed:

$$\text{2017 Honda Civic} \quad \text{Hatchback}, 5 \text{ seats} \quad \text{Frame damaged}$$

One dependency group is an exception to the above: the selection of make, model, and year of the car. Most attributes

[1] Coined by Brian Schack (Leake and Schack 2018), although we use this term differently. We use the term to indicate that these cases are not *real* — so one could say we maintain the *spirit* of the term.

[2] Another example: We should avoid generating ghost cars with mileages above the Guinness world record of 3,039,122 miles. See https://www.guinnessworldrecords.com/world-records/highest-vehicle-mileage

```
Problem Case:                          Retrieved Case:
        make_name: Mercedes-Benz               make_name: Mercedes-Benz
        model_name: E-Class                    model_name: E-Class
        year: 2012                             year: 2014
        daysonmarket: 86                       daysonmarket: 24
        mileage: 99254.0                       mileage: 76930.0
        price: ???                             price: 18825.0

        body_type: Sedan                       body_type: Sedan
        city_fuel_economy: 20.0                city_fuel_economy: 20.0
        ...                                    ...
        horsepower: 302.0                      horsepower: 302.0
        highway_fuel_economy: 30.0             highway_fuel_economy: 27.0
        listing_color: BLACK                   listing_color: GRAY
        owner_count: 3                         owner_count: 2
        wheel_system: RWD                      wheel_system: AWD
```

Figure 2: For this example car and the closest car to it, we can puzzle out the difference in price by primarily looking at the `year`, `daysonmarket`, and `mileage`

are dependent on the choice of make, model year – for example, these three attributes constrain available body types. So when altering the make, model and year, almost all other groups also need to be changed. This results in cars that are very dissimilar from the problem car, aside of course from the attributes that are completely independent from the make, model, and year (e.g. mileage). But our intuition before relies on generating *close* ghost cases! Despite this, we believe it is important to generate cars with different makes, models, and years, since this could encourage a degree of creativity in our system's retrieval ability. Of course, we hope that our similarity metric picks up the slack by only retrieving cases that are relevant to the problem.

Car Price Adaptation

The reasoning in CBR comes in the form of adaptation rules. Given a retrieved case with a known solution, these rules adapt the solution to account for the difference between this case and the problem at hand. These rules should be carefully chosen to encode the inference involved in the domain. Unfortunately, like any other knowledge engineering effort, crafting quality adaptation rules is known to be difficult. The key reason for the difficulty of adaptation is that such a rule should take us *no further* from the actual solution than the retrieved case. Because of this, it's unlikely that a wild guess at a rule will improve upon the retrieved case at all.

For adaptation using a continuous output attribute (in our case, car price), here's a helpful starting point. Let $x_1, \ldots, x_n$ each be independent attribute groups. Since $x_1, \ldots, x_n$ are independent, any change in an x_i has an independent effect e_i on the change in price. i.e.

$$\text{price}(x_1, \ldots, x_i, \ldots, x_n) = e_i(x_i, \text{price}(x_1, \ldots, x_n))$$

For example, an increase in the horsepower of a car might scale the expected price up by, say, 2%.

This gives us good intuition, but we are still left without a clear way to determine the e_i's for each attribute group x_i. The trouble here is that it is nearly impossible to disentangle the effect that each attribute group has on the final price. This frustration is well-known by the victims of knowledge engineers who, when asked "how much more expensive is a car with $150{,}000$ miles than one with $100{,}000$?", will answer that it depends on the car.

What helped us get around this was to work through adaptation by hand for a few small examples. In addition, we cannot understate the importance of finding solid rules of thumb used by experts in one's problem domain.

To illustrate our rules and how we arrived at them, consider the problem case and the retrieved closest case shown in Figure 2. Notice that the cars are very nearly identical, sans a couple of key attributes (the ellipses denote 8 attributes, all of which are identical). There are only three attributes which differ significantly between the two cars: the years of the cars, the number of days each car has been on the market, and their respective mileages. This example conveniently isolates these three factors from the rest, and so we can use it to puzzle out the difference in price.

All else being the same, we expect the 2012 car to be more expensive than the 2014 car. Similarly, we expect that more days on the market and higher mileage affect a decrease in price. We figure quantitatively how much cheaper the problem car is by using common rules-of-thumb among car dealerships. In particular, the rules we use are[3]:

1. **`year`**: Every year, the expected price of a car increases by 10%.
2. **`daysonmarket`**: The expected price drops by 0.5% every 90 days.
3. **`mileage`**: Every $10{,}000$ miles, the expected price of a car depreciates by 4%.

As an example of the kind of knowledge at play here, rule (2) accounts for the old adage that a car's price starts decreasing after it has been sitting on the lot for 90 days. Similarly, when a car has accumulated $200{,}000$ miles (a car's expected lifetime), our rules indicate that the car will be 80% off!

[3]These rules are adapted from used car blogs and inflation tools, e.g. https://www.in2013dollars.com/Used-cars-and-trucks/price-inflation/2016-to-2020

Applying these rules to our current problem, we determine that the price *decreases* by:

$$\$3765.0(\Delta\texttt{year})+\$64.9(\Delta\texttt{days})+\$1681.1(\Delta\texttt{mileage})$$

Resulting in a final price prediction of $\$13{,}314.0$ (compare that to the actual price of $\$12{,}492.0$).

Implementation of System

We implement the above ideas in a hybrid system we call CHRISTINE[4]. CHRISTINE consists of a NN and a CBR, threaded together by a method for producing the ghost base.

For our NN, we used a vanilla multi-layer perceptron regressor provided by Scikit Learn (Pedregosa et al. 2011). We used four hidden layers, and as usual fed in categorical attributes using one-hot encoding.

As for the CBR system, CHRISTINE makes use of IUCBRPy, an existing CBR library[5]. This library provides special features for linking a CBR system with a NN in a hybrid ensemble. Our CBR component uses straightforward k-nearest neighbors (k-NN) for retrieval, and its similarity metric was manually calibrated using prior knowledge about car attributes. For adaptation, CHRISTINE currently uses only the three rules-of-thumb described in the previous section.

Figure 3 illustrates how we thread these components together. Given a problem case $\vec{x}$, we generate ghost cases $\vec{a_1}, \ldots, \vec{a_n}$. We then feed these ghost cases into our pre-trained NN to obtain predictions $b_1, \ldots, b_n$. These ghost cases and their solutions are used as a case base for the CBR component, which performs retrieval and adaptation to predict the solution y.

Evaluation

Since the point of CHRISTINE's architecture is to improve on a NN's performance via adaptation, we evaluate the system on the basis of its accuracy. We compare the accuracy of CHRISTINE against the standalone components that comprise the system, i.e. we additionally test k-NN (with no adaptation), CBR, our pre-trained NN, and CHRISTINE's ghost base without adaptation.

Since many of the choices made when implementing CHRISTINE rely on special knowledge about used car pricing, we test on this single domain.

Experimental Setup

In order to compare these systems on a level playing field, we ensure that all five systems use the same training data and are tested on the same testing data. Additionally, we use the same similarity metric for k-NN, CBR, and CHRISTINE and the same adaptation rules for the latter two. Both our NN and CHRISTINE's NN share exactly the same implementation.

The original dataset we draw from is the US Used cars dataset[6]. Although this dataset includes 3 million used cars with 66 attributes each, we did not need all of these. For starters, we restrict our attention to 23 attributes (8 categorical, 15 numerical), and we filter out all cases that have an absent value for any of these attributes. We are left with $1{,}076{,}080$ rows, of which we dedicate the first $868{,}481$ for training, and the remaining $207{,}599$ for testing (we perform no shuffling on the order of these cases).

But the full training and testing sets are still too large; in order to train and test quickly (relative to the expectations of this class), we randomly select $3{,}000$ of the $868{,}481$ training cases, and use the first 100 of the $207{,}599$ testing cases. Additionally, to avoid copying the dataset redundantly, we use a shared case base with index sets that specify the training and testing sets for our five systems.

For each testing case, we predict a price using our five systems. We calculate the relative difference of each of the systems against the actual solution. The accuracy of a system is given as

$$\frac{\#\text{correct}}{\text{total}}$$

where a prediction is considered *correct* if its relative difference is within a tolerance of 20%. This might seem like a very generous tolerance, but when pricing cars we tend to consider, e.g. $12{,}000$ to be a good estimation of $10{,}000$. For a more nuanced view of each system's performance, we also include their average relative difference.

Results

	Accuracy	Average Relative Difference
k-NN (no adaptation)	53%	0.34
CBR	71%	0.23
NN	58%	0.26
CHRISTINE (no adaptation)	14%	0.61
CHRISTINE (with adaptation)	30%	0.48

Table 1: The accuracy (within a relative percent difference) and the average relative percent difference. The tolerance use to compute accuracy was 0.20.

The results are presented in Table 1. The CBR system had the highest accuracy and average relative difference, followed by the NN. CHRISTINE did not perform as well as k-NN, CBR or the NN, but CHRISTINE with adaptation does improve on itself sans adaptation.

Discussion

First of all, CBR outperforms k-NN, and CHRISTINE with adaptation outperforms CHRISTINE without. CBR additionally outperforms the NN (which makes no use of explicit domain knowledge). This suggests that our adaptation rules really do leverage knowledge about car pricing to improve the quality of the prediction.

[4]Named after the possessed car in Stephen King's novel of the same name.

[5]https://github.iu.edu/CBR-DL/IUCBRPy

[6]https://www.kaggle.com/ananaymital/us-used-cars-dataset

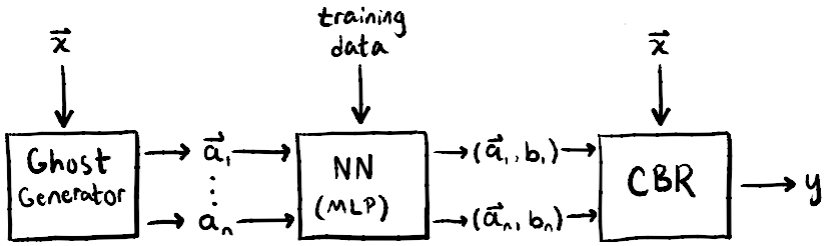

Figure 3: The architecture of CHRISTINE. The system generates ghost cars that are close to the given problem car, predicts their price, and then feeds this ghost base into CBR. The final price is obtained by doing retrieval and adaptation on this ghost base.

Unfortunately, CHRISTINE did not compete, even against k-NN. This calls into question our initial intuition about nearby ghost cases — we will discuss this further in Next Steps. For now, we will say only that the CBR's performance reassures us that adaptation is not at fault.

Strengths: Our system's primary strength is its surprisingly simple yet powerful adaptation rules. We are surprised by how far three simple rules-of-thumb have carried our CBR.

Despite CHRISTINE's poor preliminary results, our initial intuition underlying the system is still compelling. After this class is done, we plan on investigating what went wrong, largely because we both truly believe in the basic premise.

Weaknesses: Obviously CHRISTINE grossly underperformed. This is disappointing to us, but is not the end of the idea. In the Next Steps section, we will suggest what we can do differently to recover the system.

One potential point against our system is that we came up with our similarity metric somewhat naïvely. When generating ghost cases, we also perturb independent, numerical attributes naïvely. For example, we randomly generate a ghost car's mileage within some threshold of the problem's mileage, but perhaps we do not constrain this well enough.

One issue with our experimental setup is that we do not compare against a "wild guessing" price prediction system (e.g. a dummy AI that predicts an average price). Rowan suggested this when we presented. Our issue in doing this is that our problem is *regression*; normally this sort of dummy AI predicts the most common *categorical* attribute.

It was difficult to determine a reasonable threshold for accuracy. 20% seems reasonable to us for the pricing domain, but this may be unfairly bolstering each system's accuracy. We think this is fine for now, since we're primarily interested in how the systems *compare*, which is not affected too much by the threshold.

Lessons Learned

Working on this project helped us fully appreciate the difficulty in encoding domain knowledge in a knowledge-based system. We are now able to isolate exactly where we made a careful domain analysis. So where is the knowledge?

In our system, knowledge is encoded in the *similarity* metric, the *adaptation* rules, and the rules for generating valid ghost cars. When choosing weights for similarity, we often had to ask "how influential is this attribute on the price?" When designing the adaptation rules, we had to determine the precise effect of certain attributes on the change in price. And ghost cars could not be created randomly; only cars that *can* exist should be generated. This in turn demanded that we think carefully about what cars can exist. Our decisions for each of these considerations encoded, explicitly and implicitly, our knowledge about cars.

A related point is that knowledge in one of these components can be re-used in another component. For example, we had made some initial guesses for our similarity metric. But when we did a small example for adaptation, we realized that mileage drastically affects the overall price. So we revisited our choice of weights for adaptation to weight it more heavily.

Adaptation rules were particularly hard for us. While coding up adaptation rules, we were overwhelmed by the possible changes in a car as well as the changes on price these affect. Doing this properly was well beyond our knowledge of the domain. For example, to do this right we would have to consider how a Chevrolet Tahoe is considered to be a less expensive, but equivalent, Cadillac Escalade — and we would have to do this for *all* possible pairs like this.

Next Steps

There are pretty clear fixes, extensions, and improvements we can do from here. Some of these improvements are a matter of handling our data more rigorously, whereas others are attempts to recover our intuition about ghost cases.

Issues with Data

As stated earlier, we reduced the size of our data by removing any row with an empty attribute entry. Even after removing these, our dataset is still too large to run in a reasonable time. One thing we noticed later is that the data still includes significant outliers. For example, we have a 2017 Dodge Grand Caravan being grossly overpriced at $\$1{,}599{,}500$. We suspect that this example is bad data entry (since the pricing is illogical for a 3-year-old car) Additionally, there are 81 cars that are older than 1990 (oldest being 1984); these older

cases might be considered *historic* cars and should be priced differently compared to regular used cars.

So we should remove outliers of this kind, and then update our maximum and minimum values for numerical attributes. We hope this will give us more appropriate information when we normalize values. This has the potential to improve the quality of our retrieval, as well as our NN.

Additionally, we did not clean up redundant data in this dataset. For example, *Accord* and *Accord Coupe* should be considered the same car model (the word *Coupe* should actually refer to the `body_type` attribute field). This type of redudant and mis-entered data is probably causing issues in similarity and potentially adaptation. Some measures were taken to guard against this, but it is nearly impossible to programmatically determine cases like this.

We removed many attributes from consideration, usually due to the large number of values the categorical attribute could take on. Two of these we would like to reintroduce are `trim_name` and the `dealer_zip`. Both of these attributes had too many possible values, resulting in a very large one-hot encoding vector needed to pass them in to the NN. This of course affected the training time of the neural network. But both the trim name and zip code provide significant information about a car's price — for example, we can isolate the city or town where a car was sold.

We didn't test our systems on the full size of the data mainly because of the large size of the data delayed results. As expected, case retrieval takes a long time with all $868{,}481$ training cases, and the NN took over 7 hours to train. Additionally, testing $207{,}599$ test cases would have potentially taken a full year to compare all five of our systems (calculated using the time to do one iteration with the full case base). We plan on speeding the overall system up using, e.g. memoization of retrieved cases, parallelization of independent loops, and thinking carefully about our system's bottlenecks. One noticable example is that our k-NN retrieves a case, our CBR retrieves *the same case*, and then CHRISTINE retrieves *the very same case* — and we do all three calculations seperately.

Similarity and Adaptation

Creating both the similarity and adaptation for car pricing was not an exact science. For our similarity weights, we pretty much just guessed. So we expect that we will need to recalibrate our weights in the future. We also did not normalize the numerical attributes, which is a standard preprocessing step that spreads the values apart to assist k-NN.

There are also many external factors that our similarity metric doesn't account for. For example, if we were to use the zip code attribute, then we might be able to reason that cars with negative stigma in certain areas are similar. Of course, many other factors not included in our attributes (e.g. environmental considerations of the car) are not used.

In addition, the "success" of our similarity was difficult to judge. For example, the size of the case base affects which closest case is retrieved. This resulted in a tradeoff: We want a small case-base to reduce on time spent calculating similarity, but we want a large case-base in order to retrieve quality cases.

The adaptation rules we suggest here are very simple, and only focus on three key attributes. There is a vast space of variations of cars, and many car attributes are dependent on the values of the others. Because of this, we could not make adaptation rules for every variable. e.g. when adapting a make and model pair, we need to consider whether the respective brands are partners, rivals, or unrelated. Of course, because the domain is so large, we must accept that any adaptation rules we come up with are necessarily incomplete.

Before submitting this work to a conference, we plan on consulting a car salesman or mechanic about adaptation rules. We hope that such a domain expert will help us significantly improve our first try at rules.

Ghost Case Generation

The results (Table 1) raise concerns about our intiution (Figure 1). Either the general idea is wrong, our similarity metric is poor, or we generate ghost cases poorly. Since we believe in the underlying intuition about our system (and we discuss similarity above), we focus here on how we might be mis-generating ghost cases.

First, when we generate independent attributes we pick them randomly within a certain "close" threshold. For example, for mileage we generate a random value within $\pm 10{,}000$ miles of the problem case. Since mileage affects the price so significantly, this is probably generating ghost cases that are very far from the problem.

We have also reconsidered whether we should be generating cars with different makes, models, and years from the problem case. We had hoped that our similarity metric would pick up this slack, but perhaps it cannot. So we might exclude these variations on a car from the ghost base.

We are also potentially too strict with our rules about "valid cars." Namely, we refer to our case base to determine what cars are allowed (using the full $1{,}076{,}080$ data entries). But our case base does not have all possible car configurations, so our ghost case generation misses these allowed cars that are not found in the data. There is currently not a resource that was available to us that details what car configurations are possible. In addition, customizations of cars, both by manufacturer and the owner, complicate what configurations we admit.

Division of Work

Most of the code done for this project was peer-programmed. That is, we used an interactive session in Visual Studio Code, and edited program functions and classes together. In some cases, we edited code independently, but we both made sure to understand what the other person had written.

Occasionally, we trained the neural network without the other present, mainly because training was taking too long (but we could just run it while multitasking).

We developed our talk together, synchronously. To some degree, we "split" the slides depending on what we were most comfortable talking about. But we both at some point had our hands on every slide.

For the paper, we initially drafted sections independently (splitting up the work evenly). We did it this way because of

the time constraint. But at the very end, we both, together, walked through **EVERY** sentence in *excruciating* detail. :-)

References

Keane, M. T., and Kenny, E. M. 2019. How case-based reasoning explains neural networks: A theoretical analysis of xai using post-hoc explanation-by-example from a survey of ann-cbr twin-systems. In Bach, K., and Marling, C., eds., *Case-Based Reasoning Research and Development*, 155–171. Cham: Springer International Publishing.

Kolodner, J. 2014. *Case-based reasoning*. Morgan Kaufmann.

Leake, D., and Schack, B. 2018. Exploration vs. exploitation in case-base maintenance: Leveraging competence-based deletion with ghost cases. In Cox, M. T.; Funk, P.; and Begum, S., eds., *Case-Based Reasoning Research and Development*, 202–218. Cham: Springer International Publishing.

Leake, D.; Ye, X.; and Crandall, D. 2021. Supporting case-based reasoning with neural networks: An illustration for case adaptation. *Proceedings of the AAAI2021 Spring Symposium on Combining Machine Learning and Knowledge Engineering (AAAI-MAKE 20)*.

Liao, C.-K.; Liu, A.; and Chao, Y.-S. 2018. A machine learning approach to case adaptation. In *2018 IEEE First International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*, 106–109. IEEE.

Munoz-Avila, H., and Cox, M. T. 2008. Case-based plan adaptation: An analysis and review. *IEEE Intelligent Systems* 23(4):75–81.

Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; and Duchesnay, E. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12:2825–2830.

Policastro, C. A.; Carvalho, A. C.; and Delbem, A. C. 2008. adaptation-hybrid-approach. *Applied Intelligence* 28(2):101–119.

Reategui, E.; Campbell, J.; and Leao, B. 1997. Combining a neural network with case-based reasoning in a diagnostic system. *Artificial Intelligence in Medicine* 9(1):5–27. Case-based Reasoning: Opportunities and Applications in Health Care.

Rodriguez, Y.; Garcia, M. M.; De Baets, B.; Morell, C.; and Bello, R. 2006. A connectionist fuzzy case-based reasoning model. In Gelbukh, A., and Reyes-Garcia, C. A., eds., *MICAI 2006: Advances in Artificial Intelligence*, 176–185. Berlin, Heidelberg: Springer Berlin Heidelberg.